MULTI-OBJECTIVE REINFORCEMENT LEARNING FOR OPTIMAL COVID-19 CONTROL IN MICHIGAN[‡]

Alex L. Chen[§]
Electrical and Computer Engineering, MS
{lubicz}@umich.edu

Andy Chen[§]
Physics, PhD
{andych}@umich.edu

Nikhil Devraj[§]
Computer Science and Engineering, BS
{devrajn}@umich.edu

Bowei Li[§]
Civil and Environmental Engineering, PhD
{jacklbw}@umich.edu

Daniel F. Otero-Leon[§]
Industrial and Operations Engineering, PhD
{dfotero}@umich.edu

Nisarg K. Trivedi[§]
Electrical and Computer Engineering, MS
{nisargtr}@umich.edu

ABSTRACT

COVID-19 has ravaged the world over the past year and governments have struggled to stop the spread of the pandemic and prevent its debilitating societal and economic effects. They wield the power to control major operations that could help minimize losses, such as the ability to impose lockdown and distribute vaccines. However, there exist trade-offs between balancing economic and societal costs, calling for the design of policies that can balance these objectives as best as possible. In this project, we investigate how multi-objective reinforcement learning can help devise such Pareto-optimal control policies. We find that our policies have many different resulting strategies for dealing with the virus. Based on our findings, the approach in question proves to be promising for considering different ways to handle pandemics and how they may perform with respects to the economy and public health.

1 INTRODUCTION

The rapid spread of the novel coronavirus (COVID-19) has caused an unprecedented global pandemic, tremendously threatening public health and drastically affecting our society and economy (WHO, 2020). For example, during November and December of 2020, the state of Michigan repeatedly broke its record number of positive cases, eclipsing its prior peak in April seven-fold (CDC; Government). This situation calls for governing bodies to institute multiple initiatives, such as imposing lockdowns (Arango & Pelov, 2020; Wan et al., 2020) and distributing vaccines (Awasthi et al., 2020). These problems can be thought of as control problems for policymakers, begging the question: what are the optimal actions we can take to ensure success in keeping the general public healthy while also minimizing economic damage? We believe that reinforcement learning can provide insight into this problem.

We investigate how multi-objective reinforcement learning (MORL) can aid in the process of implementing government-level controls to mitigate costs due to COVID-19. We use an SIR environment to model real-life data, and then incorporate value iteration and DDQN (Double Deep Q Learning) reinforcement methods to produce policies that experts can analyze.

Contributions:

1. We implement a multi-stage, multi-objective reinforcement learning algorithm for conflicting objectives present in a real-world dataset.
2. We evaluate and analyse this complex algorithm on a COVID-19 dataset to predict the optimal lockdown strategy that optimizes public health and economic costs simultaneously.

[‡]**Project Category: New Dataset**
[§]All students contributed equally; ordered alphabetically by last name.

3. Upon examining the model's performance on real-world data, we observe that the model can substantially reduce proportion of infected people while balancing economic costs due to lockdown, proving the potential use of multi-objective reinforcement learning algorithms.
4. We introduce a visualization scheme for jointly analysing performance of the MORL algorithm on conflicting objectives.

2 Process Overview

The control problem we investigate is one that approximates a balance between societal and economic costs during the pandemic. We consider two main conflicting objectives in this project: minimizing the number of lockdowns and minimizing the number of people who become infected. Attempting to minimize the number of lockdowns imposed by policymakers is akin to attempting to reduce economic costs, because lockdowns force many citizens to lose their jobs and/or close their businesses temporarily or for good (Forsythe et al., 2020). Attempting to minimize the number of infections is a precursor to minimizing the number of deaths in a population and therefore the societal cost (Zhang et al., 2020).

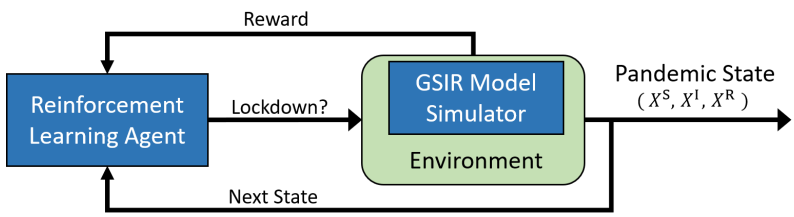


Figure 1: Control flow for optimizing one policy

Generating Policies The policies we generate control these objectives by choosing simply to enforce or lift the lockdowns. Since we have conflicting objectives, no single sequence of lockdowns will be optimal. Hence, we generate an entire set of Pareto-optimal (Stiglitz, 1981) policies and investigate how well they perform on our objectives. We employ an evolutionary algorithm using reinforcement learning to generate these policies, as discussed in section 6.

Simulating the Pandemic This problem could theoretically be handled with offline learning, since there is data available regarding how real policymakers handled the pandemic. However, those policies were not necessarily optimal (Alwan et al., 2020). In order to understand how a differing sequence of lockdowns affects the pandemic within the population, a simulation is needed. We use the Generalized Susceptible-Infected-Removed (GSIR) model in order to simulate dynamics of the pandemic (Brauer, 2008; Wan et al., 2020).

Control Scheme For each policy, one training step involves one interaction with an environment simulator that outputs the state of the pandemic in the region of interest. This feedback is used to optimize a given policy. Our control setup is demonstrated in Figure 1.

3 Pandemic Model

3.1 Generalized SIR Model

The SIR model has gained great popularity in describing the dynamics of infectious diseases (Brauer, 2008; Wan et al., 2020). At time step $t \in \mathcal{T}$, the total population of interest M can be divided into three groups: the individuals who have not been infected (susceptible) X_t^{S}, the individuals who can infect others (infected) X_t^{I}, and the quarantined or hospitalized individuals (removed) X_t^{R}. The SIR model describes how the three groups within the population evolve with t:

$$\begin{cases} X_{t+1}^{\mathrm{S}} &= X_t^{\mathrm{S}} - \beta(a_i) X_t^{\mathrm{S}} X_t^{\mathrm{I}} / M \\ X_{t+1}^{\mathrm{I}} &= X_t^{\mathrm{I}} + \beta(a_i) X_t^{\mathrm{S}} X_t^{\mathrm{I}} / M - \gamma_t X_t^{\mathrm{I}} \\ X_{t+1}^{\mathrm{R}} &= X_t^{\mathrm{R}} + \gamma X_t^{\mathrm{I}} \end{cases} \tag{1}$$

where $\beta(a_i)$ and γ are the infection and removal rate respectively. Note that $\beta(a_i)$ depends on the control action a_i. Thus, the control action heavily influences the model's predictions. In particular, this is useful because it provides us with an interactive environment to supply controls to and obtain feedback from.

However, the deterministic nature of Eq. (1) poses a great limitation when trying to model the stochastic nature of real data. Wan et al. (2020) proposes Generalized SIR model (GSIR) that introduces stochasticity into the SIR model by adding noise to each of the population group counts:

$$\begin{cases} X_{t+1}^{\mathrm{S}} = X_t^{\mathrm{S}} - e_t^{\mathrm{I}} \\ X_{t+1}^{\mathrm{R}} = X_t^{\mathrm{R}} + e_t^{\mathrm{R}} \\ X_{t+1}^{\mathrm{I}} = M - X_{l,t+1}^{\mathrm{S}} - X_{l,t+1}^{R} \end{cases} \tag{2}$$

where $e_t^{\mathrm{I}} \sim \text{Poisson}\left(\beta(a_i) X_t^{\mathrm{S}} X_t^{\mathrm{I}} / M\right)$ and $e_t^{\mathrm{R}} \sim \text{Binomial}\left(X_t^{\mathrm{I}}, \gamma\right)$.

This Generalized SIR Model is calibrated using real-world data as described in section A.2 in Appendix. Figure 3 and Table 1 in Section 7 show results of successful model calibration & verification against real-world data of Michigan state.

3.2 Data preparation

Typically COVID data reports the number of confirmed cases X_t^{C}, the positivity rate ρ_t, and the total population M. We assume the following for simplicity:

- All patients with confirmed cases were quarantined or hospitalized immediately.
- The number of confirmed cases describes the number of people who were infected within D days before the day of reporting.

The first assumption implies $X_t^{\mathrm{R}} = X_t^{\mathrm{C}}$, while the second assumption allows us to estimate X_t^{I} by $X_t^{\mathrm{I}} = M\rho_t$.

4 Problem Formulation

We use multi-objective reinforcement learning to learn controls for specific objectives that can be modified by policy-makers and governing bodies. In order to apply reinforcement learning, we define the Markov Decision Process for this scenario.

4.1 MDP Formulation

Epochs: In our approach, the agent will decide when to have a lockdown given the current SIR model state. Given the amount of current infected people at the end of the current day, the agent will decide whether or not to have a lockdown. This can also be extended to reflect the severity of the lockdown. To provide sufficient number of epochs for our model to train on, we define length of an epoch to be one week.

State Space: At the end of each day, the decision maker receives the susceptible, infected, and removed population count and makes a decision on the lockdown strictness for the next day. Thus, we define the state space as $X_t = \{X_t^I, X_t^S, X_t^R\}$ at the end of day t where X_t^I, X_t^S, and X_t^R represents the population variables mentioned before. The relationship between theses three variables allows us to reduce the state space to $X_t = \{X_t^S, X_t^I\}$ since $X_t^S + X_t^I + X_t^R = M$.

Actions: Throughout the pandemic, different decision makers define different strategies for lockdown. In general, these strategies can be thought of as levels of strictness, where a_i is the strictness level i. Therefore, we define N as the total number of lockdown strictness levels where a_0 represents no lockdown and a_N represents total lockdown. To begin with, we simplified these to two states, either full lockdown or no lockdown.

Costs: Each lockdown strictness level has different associated costs. We define c_I as the societal cost = cost of the virus on public health. This is presently measured through the number of infected people. This cost can be extended to consider healthcare, medical resources and deaths. We define $c_a(a_i)$ as the economical cost for having a lockdown of level a_i. This is presently measured based on severity of the lockdown, where the max lockdown has the maximum cost and no lockdown has the minimum cost.This cost can be extended to include unemployment rate and lost income. Total cost of taking a decision a_i in a state X_t is denoted by $c(X_t, a_i)$.

Transition probabilities: We consider two different dynamic behaviors in our model: the probability that a susceptible individual becomes an infected individual and the probability that an infected individual becomes a removed

individual. We define $P_{I_t}(I|X_t, a_i)(\beta)$ as the probability that I people get infected during a week t given that decision a_i is taken in state X_t. On the other hand, $P_{R_t}(R|X_t, a_i)(\gamma)$ denotes the probability that R people are removed from the infected group during week t given that decision a_i is taken in state X_t.

Bellman Equation: The Bellman Equation for state X_t in week t is defined as

$$V_t(X_t^I, X_t^S) = \min_{a_i} \left\{ X_t^I c_I + c_a(a_i) + \sum_{I=0}^{X_t^S} \sum_{R=0}^{X_t^I} P_{I_t}(I|X_t, a_i) P_{R_t}(R|X_t, a_i) V_{t+1}(X_t^I + I - R, X_t^S - I) \right\}$$

4.2 Multi-objective MDP

We aim to estimate two costs: a cost quantifying the infection in population and a cost quantifying economic loss due to lockdown. Several factors make estimation of both costs challenging. The societal cost representing public health needs to account for different infection levels and rates of spread. The economic cost representing economic damage needs to consider length of the lockdown period and it might accumulate or display exponential growth over time. These costs are conflict with each other so we need to design a multi-objective joint-optimization model.

First, we define $V_{I_t}(X_t)$ as the expected minimum number of infected people from day t onward and define $V_{L_t}(X_t)$ as the expected minimum number of days in lockdown from days t onward. Additionally, let us define $\omega = [\omega_I, \omega_L]$ as the weight-vector for objectives. Cumulative cost is given by:

$$V_{I,L}(X_t) = \min \{\omega_I V_{I_t}(X_t) + \omega_L V_{L_t}(X_t)\} \tag{3}$$

where $\omega_I + \omega_L = 1$.

Ideally, we can solve this problem by finding a Pareto-optimal policy Π_{t_0} defined as the policy which cannot be improved for one objective without sacrificing the others. It is important to note that these weights are unknown and may change based on situation of the infection. Considering these variables, we implemented a reinforcement learning approach to solve this problem.

5 Generating Pareto Optimal Policies: Prediction-Guided MORL

5.1 MORL Overview

We employ a variation of the prediction-guided multi-objective reinforcement learning approach proposed by Xu et al. (2020). This approach is to devise an evolutionary algorithm that evolves policies until obtaining a diverse set of Pareto-optimal policies. Figure 2 describes an overview of the evolution process.

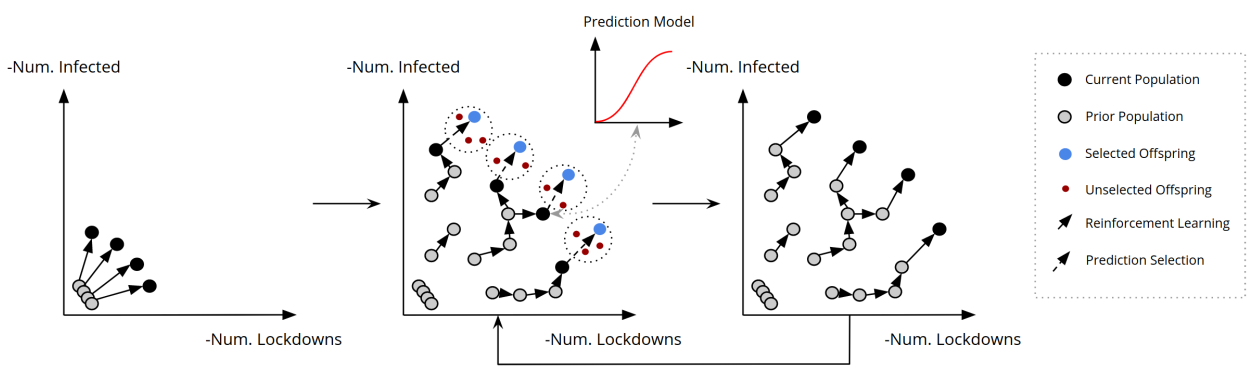


Figure 2: Overview of prediction-guided MORL algorithm. In the first stage, policies in the population are randomly initialized and trained to "warm up", so that the population is sufficiently diverse and evolution converges quickly. In the second stage, a reinforcement learning algorithm is trained for several iterations to propose a set of policies, and a prediction model predicts performance of those policies on both objectives. The population is updated based on the prediction scores in the third stage. The second and third stages are repeated for M generations to yield a set of Pareto-optimal policies. Note that the axes are negated since we intend to *minimize* lockdowns and infections.

Given an input model calibration, we can successfully initialize policies with weights $\{\boldsymbol{\omega}_i\}$ corresponding to each objective i. We have two objectives V_L and V_I with corresponding weights $\boldsymbol{\omega} = (\omega_L, \omega_I)$. We refer to a policy-weight pair $(\pi_i, \boldsymbol{\omega}_i)$ as a **task**. The overall algorithm used to generate Pareto policies is delineated in Algorithm 1.

Algorithm 1 MORL Algorithm

Inputs: # parallel-tasks n, # warm-up iterations m_w, task-iterations m_t, # generations M.
Initialize population $\mathcal{P}$, external Pareto-archive EP, RL history record $\mathcal{R}$.
▷ Warm-up Stage:
Generate task-set $\mathcal{T} = \{(\pi_i, \boldsymbol{\omega}_i)\}_{i=1}^n$ with random initial policies and evenly distributed weight-vectors.
$\mathcal{P}' \leftarrow$ Policy Selection through MOPO($\mathcal{T}, m_w, \mathcal{R}$) Algorithm 2.
Update $\mathcal{P}$ and EP using $\mathcal{P}'$.
▷ Policy Evolution Stage:
for *generation* $\leftarrow 1, 2, \ldots, M$ **do**
  $\{\Delta^i\}_{i \in |P|} \leftarrow$ Fit prediction model Δ^i for $\pi_i \in \mathcal{P}$ stored in $\mathcal{R}$
  $\mathcal{T} \leftarrow$ New prediction-guided task-set obtained using Algorithm 3.
  $\mathcal{P}' \leftarrow$ Policy Selection through MOPO($\mathcal{T}, m_t, \mathcal{R}$) Algorithm 2.
  Update $\mathcal{P}$ and EP using $\mathcal{P}'$.
end for
▷ Post-Processing and Analysis Stage:
Plot policies, rewards, Pareto-front and evaluate the policy performance.

5.2 Policy Improvement Prediction

In each generation, a set of offspring tasks is generated, in which task selection is performed. As it is time-consuming to directly perform policy optimization over all offspring tasks, we generate these tasks based on a prediction model,

$$\Delta_j^i(\omega_j) = A\frac{e^{a(\omega_j - b)} - 1}{e^{a(\omega_j - b)} + 1} + c \tag{4}$$

where $\Delta_j^i(\omega_j)$ is the predicted change of the jth objective from the ith parent task; A, a, b, and c are model parameters.

The prediction model is calibrated in each generation for each parent task by the least square regression, with the historical data that cumulated by the reinforcement learning process. Normally, multiple offspring are predicted by Eq. 4. Task selection is subsequently performed to find a set of offspring tasks that are worth optimizing.

Algorithm 2 Multi-Objective Policy Optimization - MOPO

Inputs: task-set $\mathcal{T}$, #iterations m, RL history record $\mathcal{R}$
Initialize off-spring population $\mathcal{P}'$.
for each task $(\pi_i, \omega_i) \in \mathcal{T}$ **do**
  Run policy optimization for (π_i, ω_i) for m iterations.
  Collect resultant policy π_i' in $\mathcal{P}'$.
  Store $(F(\pi_i), F(\pi_i'), \omega_i)$ in $\mathcal{R}$.
end for
Output: Offspring population $\mathcal{P}'$.

Algorithm 3 Prediction-guided Task Selection

Inputs: #tasks n, population $\mathcal{P}$, improvement prediction models $\{\Delta^i\}$, Pareto archive EP.
Initialize task-set $\mathcal{T}$ and virtual Pareto archive EP* = EP.
for $i \leftarrow 1, 2, ..., n$ **do**
  Initialize task $\mathcal{T} \leftarrow None$
  for each $\pi_i \in \mathcal{P}$ and $\omega \in$ candidate weights **do**
    if (π_i, ω) is novel and $\mathcal{Q}$(EP*,(π_i, ω)) $>$ $\mathcal{Q}$(EP*,$(\mathcal{T}_i)$) **then** $(\mathcal{T}_i) \leftarrow (\pi_i, \omega)$
    end if
  end for
  Append task $\mathcal{T}_i$ to $\mathcal{T}$.
  Update EP* by inserting the predicted offspring of $\mathcal{T}_i$).
end for
Output: Selected task-set $\mathcal{T}$.

5.3 Prediction-Guided Optimization for Task Selection

The task selection phase intends to select a set of elite tasks. The fitness of a set of selected tasks is measured by (a) hyper-volume $\mathcal{H}$ which measures how accurate and uniform the Pareto front is, and (b) sparsity $\mathcal{S}$ which indicates how uneven the distribution of tasks over the Pareto front is. Given the incoming task set $\mathcal{T} = \{(\pi_i, \omega_i)\}_{i=1}^n$, the selection is performed by solving a mixed-integer programming problem,

$$\max_{\mathcal{T}} \mathcal{Q}(\texttt{EP}, \mathcal{T}) = \mathcal{H}(\texttt{EP}, \mathcal{T}) + \alpha\mathcal{S}(\texttt{EP}, \mathcal{T}) \tag{5}$$

where $\alpha < 0$ is a hyper-parameter.

The problem is difficult to solve directly, so we adopt a greedy algorithm (Algorithm 3). We observe that this scheme can produce satisfying results without excessive computational effort for our application.

5.4 MULTI-OBJECTIVE POLICY OPTIMIZATION (MOPO)

The policy selection process identifies good policy candidates used to update the final population for a given generation. This is where our reinforcement learning implementations are useful - we iterate over a set of tasks, improve their policies through RL for a fixed number of steps, and then choose the policies whose weighted sum performance is best.

We chose to investigate two different methods for policy selection. The first method, value iteration, is implemented as a baseline method to find whether a simpler approach would arrive at a viable policy, because the present action space is finite (binary). The policies in the value iteration approach are fixed over states and varying over time. The second method learns Deep Double Q-Networks (DDQNs) to produce stochastic, state-dependent policies. The following sections describe implementation and application of both methods in context of this problem.

6 REINFORCEMENT LEARNING ALGORITHMS

In this section, we present two different approaches for the RL algorithm. The first one is Value Iteration (VI) which learns action values from policies used in previous iterations. The second approach employs a DDQN algorithm which keeps a history buffer of the past actions and results and learns the values for taking the actions on the weighted objectives.

6.1 VALUE ITERATION

The main objective of the value iteration approach is to estimate future environment behavior and choose a decision that minimizes the multi-objective cost. Nevertheless, the classical value iteration assumes an infinite time horizon, and as we are dealing with a finite time approach, we propose a correction to the algorithm as shown in Algorithm 4.

Algorithm 4 Value Iteration for Finite Time Horizon

procedure VL(ω_I,ω_L,X_I,X_S)
  m : Total number of simulated steps
  n : Total number of iterations
  π: Size m policy, for which π_t is the chosen action for step t.
  for $i \in \{1, \ldots, n\}$ **do**
    $X_{I_L}, X_{S_L}, X_{I_N}, X_{S_N} \leftarrow X_I, X_S, X_I, X_S$
    for $j \in \{1, \ldots, m\}$ **do**
      $X_{I_L}, X_{S_L} \leftarrow$ getNextStates$(X_{I_L}, X_{S_L}, 1)$
      $X_{I_N}, X_{S_N} \leftarrow$ getNextStates$(X_{I_N}, X_{S_N}, 0)$
      $V_L \leftarrow \omega_I X_{I_L} + \omega_L$
      $V_N \leftarrow \omega_I X_{I_N}$
      $X_{I_{L_K}}, X_{S_{L_K}}, X_{I_{N_K}}, X_{S_{N_K}} \leftarrow X_{I_L}, X_{S_L}, X_{I_N}, X_{S_N}$
      for $k \in \{j+1, \ldots, m\}$ **do**
        $X_{I_{L_K}}, X_{S_{L_K}} \leftarrow$ getNextStates$(X_{I_{L_K}}, X_{S_{L_K}}, \pi_k$ if $i \neq 1;$ else $1)$
        $X_{I_{N_K}}, X_{S_{N_K}} \leftarrow$ getNextStates$(X_{I_{N_K}}, X_{S_{N_K}}, \pi_k$ if $i \neq 1;$ else $0)$
        $V_L \leftarrow V_L + \omega_I X_{I_{L_K}} + \omega_L$
        $V_N \leftarrow V_N + \omega_I X_{I_{N_K}}$
      end for
      $\pi_j \leftarrow \arg\min\{V_L, V_N\}$
    end for
  end for
  return π
end procedure

The algorithm receives as input the following parameters: ω_I as the weight for the objective of minimizing infected population, ω_L as the weight for the objective of minimizing the number of lockdowns, X_I as the size of the infected population, and X_S as the size of the susceptible population.

To initialize the algorithm, on the first iteration, we assume that if a decision is chosen on a specific step, the same decision is selected for the next steps. We then estimate the multi-objective values and evaluate the *argmin*. For the

next steps, the algorithm learns from the previous step, and instead of choosing the same policy to get the following states, we use the policy from the last iteration.

6.2 DDQN

Here we employ Deep Double Q-Networks (DDQNs) to learn policies. Each DDQN takes two inputs (X_t^S, X_t^I) and passes them through two hidden layers each with 16 neurons. The output then estimates the Q values for each action. From this output we choose the action that maximizes the objective. The objectives represent the negated societal and economic costs. The environment produces separate rewards corresponding to each objective which are then scaled to ensure proportionate magnitudes of both objectives. Our DDQN then receives this reward during training, and scales them according to its weighted objectives to update the estimated Q table. The essential algorithm follows the outline presented in Algorithms (1, 2, 3) with MOPO customized for our problem.

7 Results

Our approach is meant to recommend a set of policies that would help optimize the number of infections and the number of lockdowns concurrently. Here we display results regarding our GSIR model's effectiveness and regarding both approaches and some interesting analysis.

7.1 GSIR Model

Based on the data provided by the state of Michigan Government, the GSIR model is calibrated based on the section A.2. The comparison between the real data and the model is shown in Fig. 3. We can see that the model is sufficiently accurate. The calibrated parameters are summarized in Table 1, where $\hat{\beta}(0)$ and $\hat{\beta}(1)$ correspond to no restrictions and full lockdown periods, respectively. It is interesting to note that $\hat{\beta}(1)/\hat{\gamma} > 1$ while $\hat{\beta}(0)/\hat{\gamma} < 1$, which implies that the pandemic explodes with no restrictions, and will decay if full lockdown is enforced.

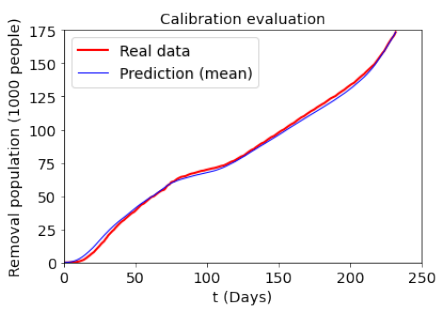


Figure 3: Comparison between the real data and the model

Table 1: Calibrated GSIR parameters

$\hat{\gamma}$	$\hat{\beta}(0)$	$\hat{\beta}(1)$
0.1062	0.1236	0.1048

7.2 Value Iteration

After running the algorithm, we obtained 10 different tasks for the Pareto front. Each task represents a set of weights for the objectives. We ran the algorithm for 50 time steps (representing days) and compared the resulting policies to the policy employed by government in Michigan.

Figure 4 presents each of the resulting policies for ten tasks. If a task has higher angle, then the weight to minimize infected population is higher. As the weight gets smaller, fewer lockdowns are performed because a higher weight is assigned to minimizing lockdowns. Given this, it clearly shows a threshold where the increase in infected people is much higher than the benefits of not having the lockdown. Therefore, most of the tasks weigh 0.5 or more. Nevertheless, it is worth noticing how the last two tasks (with the smaller angles) behave; they focus more on the last 25 days. On the other hand, you can see that the other tasks have lockdowns in most early 25 days.

We then tested the policies versus the original policy that Michigan had from different periods. We tested since day 50 of the lockdown, where Michigan was in lockdown for 42 days, since day 70 where Michigan was in lockdown for

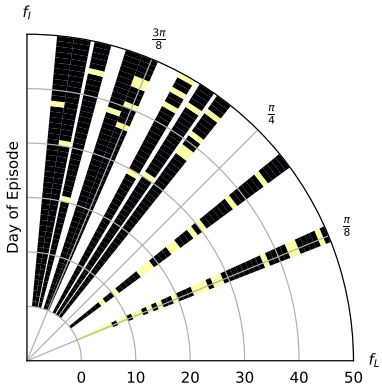
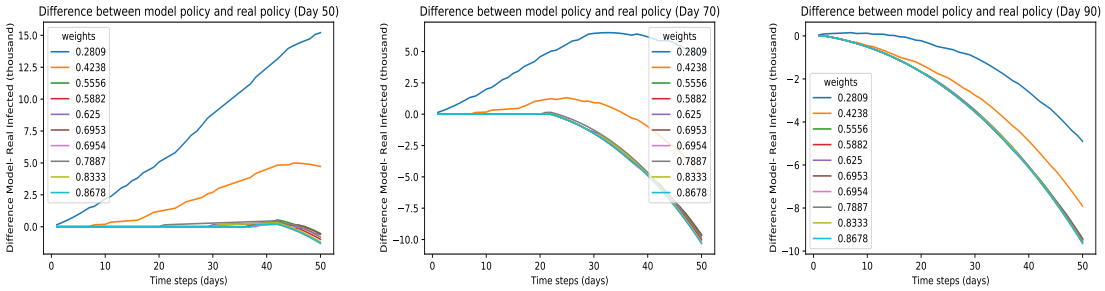


Figure 4: We present the policies for the resulting tasks $(\pi_i, \boldsymbol{\omega}_i)$, represented by the weight of the infected population objective. Each ray in the graph has a specific policy plotted on it at an angle of $\tan^{-1}(\frac{\omega_I}{\omega_L})$, where $\boldsymbol{\omega}_i = (\omega_I, \omega_L)$. At a higher angle ω_I is higher. Every black cell corresponds to a lockdown and yellow cells correspond to lifted lockdowns. It is worth noting that when the weight for the number of infected people increases, more lockdowns are imposed, given that higher priority is given to minimizing the number of infected people.

22 days, and day 90 where Michigan was in lockdown for 2 days. We calculated the difference between our model's estimate of the infected population versus the real Michigan infected population and displayed the results in Figure 5.



(a) Best case policy of the model reduces total number of infected people by 1,000.

(b) Best case policy of the model reduces total number of infected people by 10,000.

(c) Best case policy of the model reduces total number of infected people by 10,000.

Figure 5: We tested our model in three different scenarios (since day 50, 70 or 90 of the pandemic). In the y-axis we present the difference in the infected population (in thousands) between our policy and the actual (governed) policy. In general, policies with weighing infections higher than 0.55 perform better than the actual policies.

For all three cases, at least one of our resulting policies performed better than the real policy. It is interesting to see that our policies tend to minimize in the long run the number of infected rather than in each step. This behavior shows that we may have more infected on some days than the real data, but in the end, we reduce the total number of the infected population. Also, it is essential to remark that the maximum reduced size of the infected population does not change between Figures 7b and 7c. But if we compare the policy for weight 0.42, we can see that we pass from reducing the number of infected from by 5000 to almost by 8000.

To summarize, we believe that for Michigan, we need to set similar weights to minimize each objective. Nevertheless, when giving a lower weight to minimizing the infected population, some cases may also generate a reduction.

7.3 DDQN

In our second method, we obtained multiple tasks for the Pareto front. Similar to value iteration, we ran the algorithm for 50 time steps(days) and can compare the results to the Michigan policy.

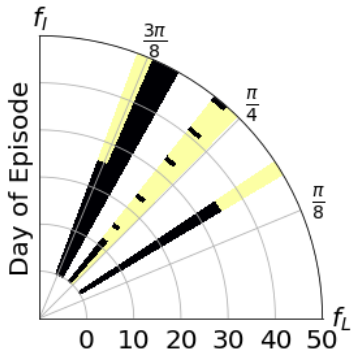
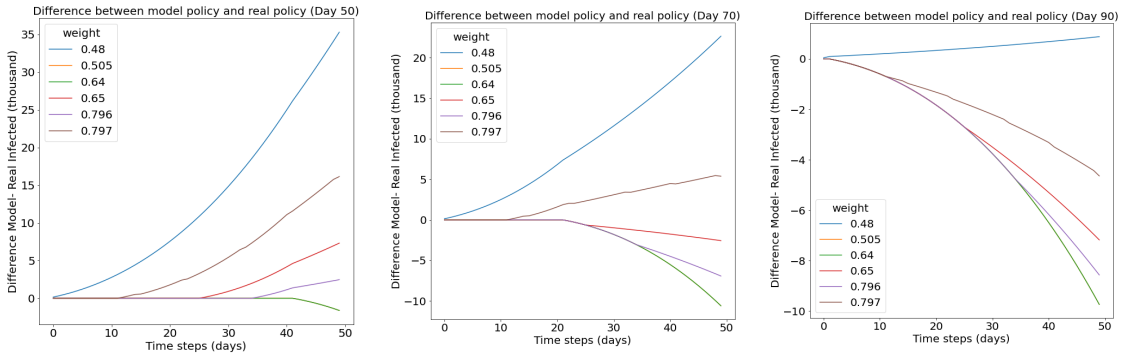


Figure 6: Similar to Figure 4, we plot the policies for six different resulting tasks.

Figure 6 shows how the weights for each objective results in different policies. As with value iteration, we see that as the weight for economic cost goes up, less lockdowns are used generally. However, because we use a DDQN network that is having its reward weights changed while training, it has a bit more unpredictable results with respect to objective weights. We would expect that the higher the objective weight for societal cost, the more lockdowns we would have. However, we can see this is only usually true, but not always true. This is because our DDQN may have been optimized for a prior objective weight pair and has not yet been fully trained for the current weight pair. Increasing the number of DDQN updates shows better performance, at the expense of much longer training time before decisions can be made.

We can see some interesting policies being generated. For example, earlier lockdown is always done, and then intermittent or no lockdown is done. This seems similar to current methods around the world. We can also see from Figure 7 that the success of each of these policy networks depends on the time-frame, and they generally perform better with respect to number of infections, on later starting time when society was less inclined to do lockdowns.



(a) Best case policy of the model reduces total number of infected people by 1,000.

(b) Best case policy of the model reduces total number of infected people by 12,000.

(c) Best case policy of the model reduces total number of infected people by 10,000.

Figure 7: We tested our DDQN model with three different scenarios (since day 50, 70 or 90 of the lockdown),

7.4 Pareto Optimality

To achieve a higher granularity for policies that are possibly Pareto optimal, the layer weights of the Pareto policy DDQNs are interpolated to form a Pareto front. The Pareto policies are first ordered according to the societal cost f_I and policy pairs are established between adjacent policies. The neural network weights for each policy in a pair are taken and 100 interpolated policies are created with neural network weights evenly spaced between the weight values of the Pareto pair. The interpolated policies are evaluated to obtain the societal cost f_I and economic cost f_L, and each Pareto and interpolated policy is represented by the cost value pair in Figure 8. The Pareto policies are indicated in large blue dots while the interpolated policies are indicated in small red dots. The concave curvature and evenly spaced nature of the interpolated policy form the smooth Pareto front that is expected.

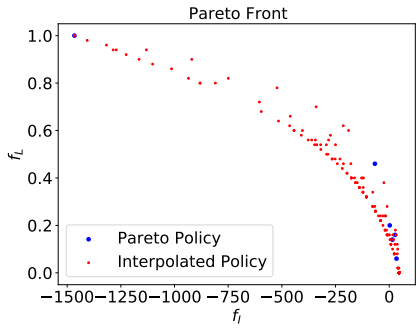


Figure 8: DDQN Pareto Front. The Pareto policies from the last evolution stage are represented in blue dots while the policies with model parameters interpolated between two Pareto policies are represented in red dots.

8 Discussion

Analysis Our results demonstrate how we can devise policies in order to optimize both societal and economic costs. Figures 4 and 6 show examples of policies that help minimize lockdowns and infections simultaneously through different RL approaches. We observe that the DDQN approach is successful in not only minimizing the number of lockdowns, but it's also better than the value iteration approach in avoiding short periods of lockdowns (such as single-day lockdowns). Furthermore, the resultant policies generally succeed in avoiding a sustained lockdown over very long periods of time.

Both of our approaches decrease the size of the infected population. In some cases, the value iteration approach outperforms the DDQN approach, as for lower weights (0.42) VI has a higher decrease in infected population than DDQN. This results in fewer lockdowns throughout the time horizon. Nevertheless, value iteration proposes a policy that changes with respect to time and not with respect to states. This policy lacks the flexibility that the DDQN policy has, affecting the results if the environment behavior changes drastically. Additionally, the VI policies tend to have intermittent lockdowns, which are difficult to be applied practically.

We also observe that the policies from our DDQN approach afford a continuous Pareto front. In a more robust setting, such a Pareto front could potentially serve as a reference for policymakers to sample their lockdown strategies from. They could weigh tradeoffs proposed by the Pareto front and choose policies that best fit their specifications.

Finally, we highlight the way our model performed with respect to unseen pandemic data in the state of Michigan. Figures 5 and 7 both demonstrate how our methods learn policies that reduce the number of infected more rapidly than those implemented in the real world. Additionally we see that the economic damage of lockdowns is also reduced by minimizing time in lockdowns. This is our main indicator that the proposed method is promising.

Future Work In the future we would like to train and test our model on data from other regions of the world. This would help us understand how our policies work in varying data distributions. It may also be useful in adjusting our RL approaches to consider a larger state space that includes environmental factors of regions, such as population densities and global health indices.

We would also like to test with more levels of lockdown severity. Currently we only decide between some sort of lockdown or no lockdown. This leaves a lot of uncertainty with respects to what levels or kinds of lockdowns should be used. By including more action levels we can give better predictions to those that would want to use our methods to influence decisions. Additionally lockdowns are generally not done on a day to day basis like we propose. It would be worthwhile to introduce an additional cost to switching back and forth too quickly between lockdown and no lockdown.

Another possible direction is to test more powerful RL algorithms for our policy selection, such as actor-critic methods. This would be more promising as we increase the size of the state and action spaces.

Another large improvement we could make is also to expand the SIR model. There are multiple expansions such as the SEIR model, MSIR model, and SEIS model. These models may allow for more accurate analysis of the state, which would result in better policies.

REFERENCES

Nisreen A Alwan, Rochelle Ann Burgess, Simon Ashworth, Rupert Beale, Nahid Bhadelia, Debby Bogaert, Jennifer Dowd, Isabella Eckerle, Lynn R Goldman, Trisha Greenhalgh, Deepti Gurdasani, Adam Hamdy, William P Hanage, Emma B Hodcroft, Zoë Hyde, Paul Kellam, Michelle Kelly-Irving, Florian Krammer, Marc Lipsitch, Alan McNally, Martin McKee, Ali Nouri, Dominic Pimenta, Viola Priesemann, Harry Rutter, Joshua Silver, Devi Sridhar, Charles Swanton, Rochelle P Walensky, Gavin Yamey, and Hisham Ziauddeen. Scientific consensus on the COVID-19 pandemic: we need to act now. *The Lancet*, 396(10260):e71–e72, October 2020. doi: 10.1016/s0140-6736(20)32153-x. URL https://doi.org/10.1016/s0140-6736(20)32153-x.

Mauricio Arango and Lyudmil Pelov. Covid-19 pandemic cyclic lockdown optimization using reinforcement learning. *arXiv preprint arXiv:2009.04647*, 2020.

Raghav Awasthi, Keerat Kaur Guliani, Arshita Bhatt, Mehrab Singh Gill, Aditya Nagori, Ponnurangam Kumaraguru, and Tavpritesh Sethi. Vacsim: Learning effective strategies for covid-19 vaccine distribution using reinforcement learning. *arXiv preprint arXiv:2009.06602*, 2020.

Fred Brauer. Compartmental models in epidemiology. In *Mathematical epidemiology*, pp. 19–79. Springer, 2008.

CDC. Cdc covid data tracker. URL https://covid.cdc.gov/covid-data-tracker/#cases_casesper100klast7days.

Eliza Forsythe, Lisa B. Kahn, Fabian Lange, and David G. Wiczer. Searching, Recalls, and Tightness: An Interim Report on the COVID Labor Market. NBER Working Papers 28083, National Bureau of Economic Research, Inc, November 2020. URL https://ideas.repec.org/p/nbr/nberwo/28083.html.

Michigan Government. Coronavirus / michigan data. URL https://www.michigan.gov/coronavirus/0,9753,7-406-98163_98173---,00.html.

Joseph E. Stiglitz. Pareto optimality and competition. *The Journal of Finance*, 36(2):235–251, 1981. ISSN 00221082, 15406261. URL http://www.jstor.org/stable/2327006.

Runzhe Wan, Xinyu Zhang, and Rui Song. Multi-objective reinforcement learning for infectious disease control with application to covid-19 spread. *arXiv preprint arXiv:2009.04607*, 2020.

WHO. Coronavirus disease (covid-19): weekly epidemiological update. 2020.

Jie Xu, Yunsheng Tian, Pingchuan Ma, Daniela Rus, Shinjiro Sueda, and Wojciech Matusik. Prediction-guided multi-objective reinforcement learning for continuous robot control. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

Renyi Zhang, Yixin Li, Annie L. Zhang, Yuan Wang, and Mario J. Molina. Identifying airborne transmission as the dominant route for the spread of covid-19. *Proceedings of the National Academy of Sciences*, 117(26):14857–14863, 2020. ISSN 0027-8424. doi: 10.1073/pnas.2009637117. URL https://www.pnas.org/content/117/26/14857.

A Appendix

A.1 Source Code

Our source code is located here: github.com/nik7273/covid-pgmorl

A.2 GSIR model calibration

To calibrate the GSIR, the generalized linear model is considered:

$$\mathbb{E}[Y] = g^{-1}(Z\theta) \tag{6}$$

where Y is output; Z is the input vector; θ is the parameter vector; g is the link function.

$$\text{Var}[Y] = V(g^{-1}(Z\theta)) \tag{7}$$

For X^{S}, define model input and output respectively as $Z^{\text{S}} = X_t^{\text{S}} X_t^{\text{I}}/M$ and $Y^S = X_t^{\text{S}} - X_{t+1}^{\text{S}}$.

$$\begin{cases} Y^S \sim \text{Poisson}\left(\beta(a_i) Z^{\text{S}}\right) \\ \mathbb{E}[Y^S | Z^{\text{S}}] = \beta(a_i) Z^{\text{S}} \end{cases} \tag{8}$$

Comparing the expectation with Eq. (6), one sees that the identity link function would be the most straightforward. Note that only data within the period of action a_i is taken is considered. The same procedure is followed for β with all other actions. For clarity, $\beta(a_i)$ is denoted as β in the rests of this section.

Calibrate the model by leverage the maximum likelihood estimation. Consider Poisson regression by a set of data $\mathcal{D}^{\text{S}} = \{Z_k^{\text{S}}, Y_k^{\text{S}} : k = 1, 2, ..., m\}$, the log-likelihood reads:

$$\mathcal{L}(\beta|\mathcal{D}^{\text{S}}) = \log\left(\prod_{k=1}^{m} \frac{\left(\beta Z_k^{\text{S}}\right)^{Y_k^S}}{Y_k^S!} e^{-\beta Z_k^{\text{S}}}\right) = \sum_{k=1}^{m} \left[Y_k^S \log\left(\beta Z_k^{\text{S}}\right) - \beta Z_k^{\text{S}} - \log(Y_k^S!)\right] \tag{9}$$

The maximum likelihood estimation is:

$$\hat{\beta} = \arg\max\left\{\mathcal{L}(\beta|\mathcal{D}^{\text{S}})\right\} \tag{10}$$

Based on the optimality conditions,

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \beta} & = \sum_{k=1}^{m} \left(\frac{Y_k^S}{\beta} - Z_k^{\text{S}}\right) = 0 \\ \frac{\partial \mathcal{L}^2}{\partial^2 \beta} & = -\frac{1}{\beta^2} \sum_{k=1}^{m} \frac{Y_k^S}{Z_k^{\text{S}}} < 0 \end{cases} \tag{11}$$

$$\text{Thus,} \quad \hat{\beta} = \frac{\sum_{k=1}^{m} Y_k^S}{\sum_{k=1}^{m} Z_k^{\text{S}}} \tag{12}$$

For X^{R}, define model input and output respectively as $Z^{\text{R}} = X_t^{\text{I}}$ and $Y^{\text{R}} = X_{t+1}^{\text{R}} - X_t^{\text{R}}$,

$$\begin{cases} Y^{\text{R}} \sim \text{Binomial}\left(Z^{\text{R}}, \gamma\right) \\ \mathbb{E}[Y^{\text{R}} \sim \text{Binomial} | Z^{\text{R}}] = \gamma Z^{\text{R}} \end{cases} \tag{13}$$

Consider the identity link function. Note that, since actions does not influence γ, all data can be involved in fitting the γ. Given a set of data $\mathcal{D}^{\text{R}} = \{Z_k^{\text{R}}, Y_k^{\text{R}} : k = 1, 2, ..., m\}$, the log-likelihood reads:

$$\mathcal{L}(\gamma|\mathcal{D}^{\text{R}}) = \log\left(\prod_{k=1}^{m} \binom{Z_k^{\text{R}}}{Y_k^{\text{R}}} \gamma^{Y_k^{\text{R}}} (1-\gamma)^{Z_k^{\text{R}} - Y_k^{\text{R}}}\right) = \sum_{k=1}^{m} \left[\log\binom{Z_k^{\text{R}}}{Y_k^{\text{R}}} + Y_k^S \log(\gamma) + (Z_k^{\text{R}} - Y_k^{\text{R}}) \log(1-\gamma)\right] \tag{14}$$

The maximum likelihood estimation is:

$$\hat{\gamma} = \arg\max\left\{\mathcal{L}(\gamma|\mathcal{D}^{\text{R}})\right\} \tag{15}$$

Based on the optimality conditions,

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial \gamma} & = \sum_{k=1}^{m} \left(\frac{Y_k^R}{\gamma} - \frac{Z_k^{\mathrm{R}} - Y_k^{\mathrm{R}}}{1-\gamma} \right) = 0 \\ \frac{\partial \mathcal{L}^2}{\partial^2 \gamma} & = -\sum_{k=1}^{m} \left(\frac{Y_k^R}{\gamma^2} + \frac{Z_k^{\mathrm{R}} - Y_k^{\mathrm{R}}}{(1-\gamma)^2} \right) < 0 \end{cases} \tag{16}$$

$$\text{Thus,} \quad \hat{\gamma} = \frac{\sum_{k=1}^{m} Y_k^R}{\sum_{k=1}^{m} Z_k^{\mathrm{R}}} \tag{17}$$